

CUESTIONARIO TEÓRICO - López Franco - Pietras Matias

Las siguientes preguntas deben contestarse y entregarse en un informe aparte.

1) Sistemas Operativos - Generalidades

a) ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?

Es el scheduler y su función principal es la de organizar las diferentes tareas y otorgarles un tiempo para que estas se ejecuten.

b) Explique a que se denomina “Contexto de Ejecución”. ¿Cuáles son las variables intervinientes?

Cada tarea tiene asociado un contexto el cual está formado por los registros internos del microprocesador, el contador del programa y la pila; que se usa para la llamada a funciones y para almacenar las variables locales.

c) ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real?

Se denomina Sistema Operativo de Tiempo Real a un sistema que hace uso de múltiples hilos de ejecución por medio de un solo procesador, pudiendo así realizar múltiples tareas en paralelo.

Se desea usar un Sistema Operativo de Tiempo Real en casos en los que se requiera que el procesador realice varias acciones “al mismo tiempo” dándole a cada acción una tarea, las cuales el Scheduler se encarga de gestionar.

2) Kernel de FreeRTOS

a) Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.

Una Tarea es un pequeño programa, tiene un punto de entrada y es básicamente un loop infinito. Ninguna tarea debería retornar a su función de implementación. Y si una tarea ya no es necesaria, esta podrá ser borrada.

```
void storage_task(void *params)
```

```
{
```

```
    //Contenido de la Tarea
```

```
while(1){
```

```
    //Lógica a ejecutar
```

```
}
```

```

}

main()

{

xTaskCreate(storage_task, "storage_task", configMINIMAL_STACK_SIZE*2, NULL,
2,NULL);

vTaskStartScheduler();

}

```

b) Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.

Una tarea puede estar Ready, Running, Suspended o Blocked.

Ready: Lista y esperando por el scheduler para iniciar.

Running: es cuando una tarea está activa y corriendo.

Blocked: una tarea está bloqueada cuando por ejemplo está esperando por un evento, que puede ser de dos tipos: Temporales (Retardo) o de sincronización (bloquearse para esperar datos de una cola).

Suspended: Es un estado de no ejecución, para que el scheduler no ejecute estas tareas, para salir de este estado se debe hacer un llamado a la función vTaskResume().

Transiciones:

Suspended

- Para que una tarea pase al estado Suspended se debe llamar a la función vTaskSuspend(), esta misma puede ser llamada por otras tareas enviándole el Handler de la tarea a suspender.
- Para que una tarea pase al estado Ready se debe llamar a la función vTaskResume(), esta misma puede ser llamada por otras tareas enviándole el Handler de la tarea a cambiar de estado.

Blocked

- Solo se puede bloquear una Tarea si esta antes estaba en estado de Running y se bloquea debido a la espera de un evento.
- De Blocked se puede ir a ready, a través del evento o se puede ir a Suspended a través de vTaskSuspend()

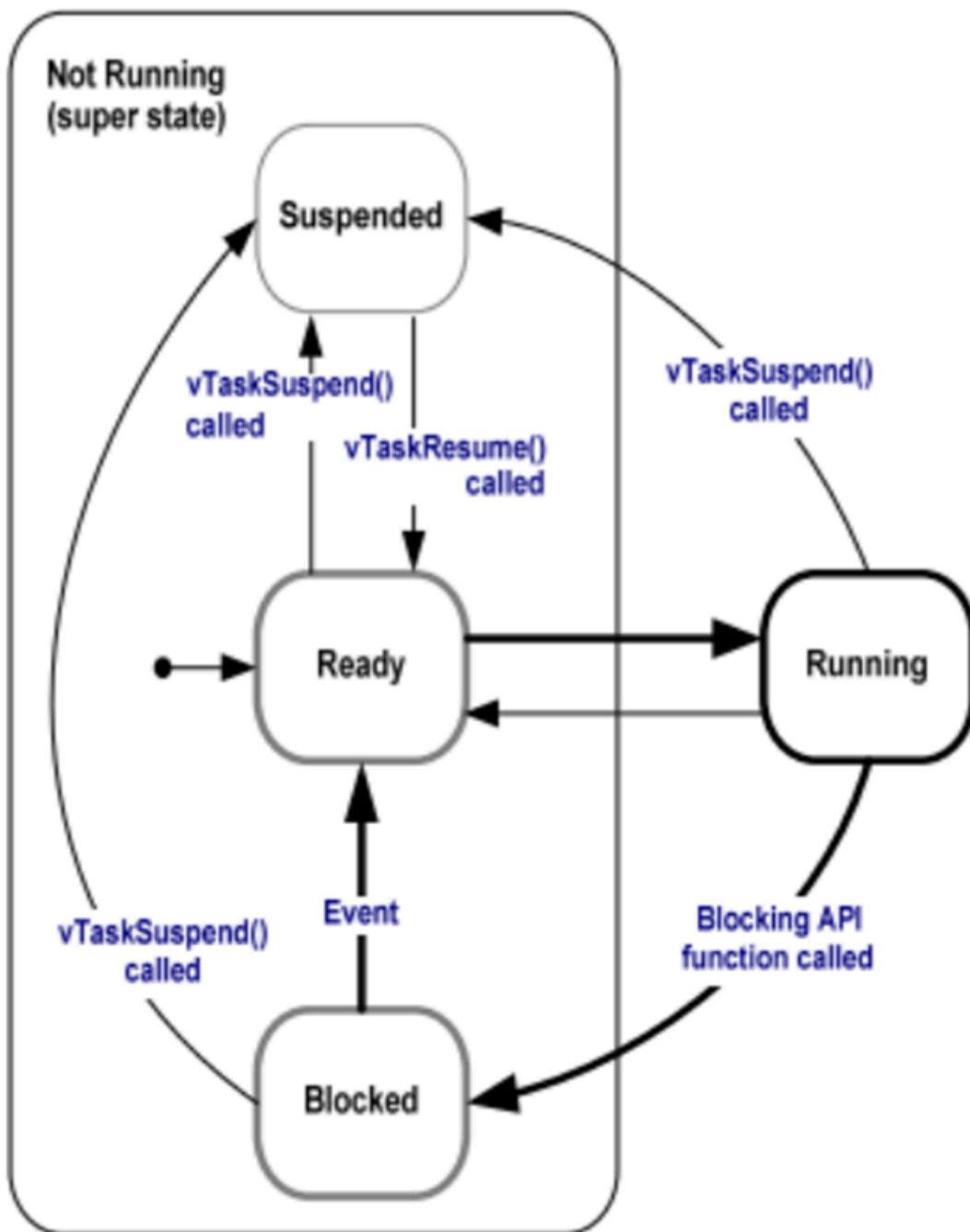
Ready

- Para ir a Ready se puede ir desde los tres estados, desde running cuando termina de ejecutarse la tarea en sí, desde Suspended a través de un vTaskResume() o desde Blocked a través de un evento.

- De Ready se puede ir a Running cuando la tarea es llamada por el Scheduler o puede ir a Suspended a través de `vTaskSuspend()`.

Running

- Desde Running se puede ir a los tres estados, suspended a través de un `vTaskSuspend()` a Blocked por medio de un evento (que haya que esperar) o a Ready cuando se termine la tarea en ejecución.
- Y solo se puede ir a Running desde Ready cuando la tarea sea llamada por el Scheduler.



c) ¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”? ¿y a que nos referimos con el concepto de “tareas manejadas por eventos”?

Son tareas en las cuales no se hacen llamados a funciones API que pudieran hacer que las tareas se bloqueen, por lo que estas tareas siempre estarán ejecutándose o disponibles para ejecutarse.

Una tarea manejada por evento solo tiene trabajo (o proceso) para realizar cuando haya ocurrido un evento que la disparada, y no puede entrar en ejecución sin que ocurra ese evento.

d) ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.

Una Tarea en estado Blocked puede estar esperando por dos tipos de eventos diferentes: Temporales o de sincronización.

- Temporales: Consisten en un cierto retardo de tiempo que debe cumplirse, por ejemplo, una espera de 10 milisegundos.
- Sincronización: El evento proviene de otra tarea o interrupción, por ejemplo, una tarea se bloquea para esperar a que entren los datos de una cola.

e) ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

Idle Task o la Tarea Ociosa es una tarea que se crea automáticamente cuando se llama a la función `vTaskStartScheduler()`, esta no hace nada más que estar en bucle y siempre es capaz de ejecutarse, siempre teniendo una prioridad lo más baja posible (de 0).

No, el procesador siempre necesita algo para ejecutar, ósea que siempre debe de haber al menos una tarea en ejecución, para ello se crea la Idle Task.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

Una cola puede contener un número finito de elementos de datos, de tamaño fijo y del mismo tipo. El número máximo de elementos que una cola puede contener se llama su "longitud". Tanto la longitud y el tamaño de cada elemento de datos se establecen cuando la cola es creada. Normalmente las colas son usadas como buffers primero en entrar-primero en salir (FIFO) donde los datos se escriben en el final de la cola y se retira de la parte frontal (cabeza) de la cola. También es posible escribir en la parte delantera de una Cola.

La escritura de datos a una cola se lleva a cabo copiando byte por byte de los datos a ser almacenados en la cola en sí misma. La lectura de datos de una cola provoca una copia de los datos que se eliminan de la cola.

4) Explique utilizando un ejemplo con código él porque es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

Es común que el recurso Cola posea múltiples tareas escritoras mientras que solo tenga una sola tarea lectora debido a que por ejemplo se puede contar con múltiples sensores los

cuales guardarán sus mediciones en la cola, mientras que para trabajar o mostrar esos datos puede ser que se necesite de una tarea solamente, como lo puede ser una tarea que controle un display para mostrar los datos obtenidos.

```
#include <FreeRTOS.h>

#include <task.h>

#include <queue.h>

#include <semphr.h>

QueueHandle_t q_lux;          // Cola para comunicación entre tareas

void get_lux_from_bh1750(void *pvParameters) {

    float lux = 0;

    while (1) {

        bh1750_get_lux();

        xQueueSend(q_lux, &lux, portMAX_DELAY);

        vTaskDelay(pdMS_TO_TICKS(500));

    }

}

void get_lux_from_temt6000(void *pvParameters) {

    float lux;

    while (1) {

        lux = temt6000_get_lux();

        xQueueSend(q_lux, &lux, portMAX_DELAY);

        printf("[BAJA] Mutex liberado.\n");

        vTaskDelay(pdMS_TO_TICKS(2000));

    }

}

void show_lux(void *pvParameters){

    float lux = 0;

    while(1){
```

```

        xQueueReceive(q_lux, &lux, portMAX_DELAY);

        printf("lux: %.2f \n", lux);

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void app_main() {

    xQueue = xQueueCreate(5, sizeof(float));

    // Crear tareas

    xTaskCreate(get_lux_from_temt6000, "get_lux_from_temt6000",
2048, NULL, 1, NULL);

    xTaskCreate(get_lux_from_bh1750, "get_lux_from_bh1750", 2048,
NULL, 1, NULL);

    xTaskCreate(show_lux, "show_lux", 2048, NULL, 1, NULL);

    // Iniciar scheduler

    vTaskStartScheduler();

}

```

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM?

Si se desea enviar datos compuestos como un int y un float al mismo tiempo se podría generar un struct el cual contenga ambos datos y de esa forma poder enviar datos “compuestos” sin generar conflictos.

Si los datos fueran “grandes” lo más recomendable sería, en vez de enviar los datos, enviar punteros hacia los datos, lo cual es más eficiente en el tiempo de procesamiento y la cantidad de RAM necesaria para crear la cola.

Y se deben tener en cuenta los dos siguientes puntos:

1. El propietario de la RAM siendo apuntado claramente definido cuando se comparte RAM entre tareas se debe tener cuidado de que las tareas no modifiquen la RAM al mismo tiempo, lo que podría hacer que el contenido de la memoria pase a ser inválida o inconsistente.
2. La memoria RAM siendo apuntada sigue siendo válida cuando la RAM haya sido apuntada, la tarea que la apunto debe también liberarla, ósea que ninguna de las otras tareas debe poder acceder a la memoria mientras esta está siendo usada.

6) Explicar el funcionamiento del esquema de la fig 1:



Fig. 1. Recibir datos de múltiples fuentes

En el esquema anterior se muestra una cola con múltiples entradas y una sola salida, con el detalle de que los emisores no envían cualquier tipo de dato, sino que envían un dato del tipo struct. Esto con el objetivo de, aparte de enviar datos, se envía de donde provienen estos datos, ya que dependiendo de su origen se determinará cómo deben ser procesados por el controlador.

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

```

SemaphoreHandle_t mutex;

int g_count=0; // Recurso a resguardar

void task_b(void *params){
    for(;;){
        xSemaphoreTake(mutex, portMAX_DELAY);
        g_count++; // Uso de recurso
        xSemaphoreGive(mutex);
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}

void task_a(void *params){
    for(;;){
        xSemaphoreTake(mutex, portMAX_DELAY);
        g_count--; // Uso de recurso
        xSemaphoreGive(mutex);
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}
  
```

```

int main(){

mutex = xSemaphoreCreateMutex();

xTaskCreate(
    task_a,
    "Tarea A",
    configMINIMAL_STACK_SIZE,
    NULL,
    tskIDLE_PRIORITY+1,
    NULL
);

xTaskCreate(
    task_b,
    "Tarea B",
    configMINIMAL_STACK_SIZE,
    NULL,
    tskIDLE_PRIORITY+1,
    NULL
);

vTaskStartScheduler();
}

```

8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

```

#include <FreeRTOS.h>

#include <task.h>

#include <queue.h>

#include <semphr.h>

// Recursos compartidos

SemaphoreHandle_t xMutex; // Mutex para evitar condiciones de
carrera

QueueHandle_t xQueue;      // Cola para comunicación entre tareas

// Tarea de ALTA prioridad (debe ejecutarse rápido)

void low_priority_task(void *pvParameters) {

    while (1) {

        printf("[ALTA] Esperando el mutex...\n");

        xSemaphoreTake(xMutex, portMAX_DELAY); // Bloquea hasta
obtener el mutex
    }
}

```



```

        printf("[ALTA] Recurso adquirido!\n");

        vTaskDelay(pdMS_TO_TICKS(100)); // Simula uso del recurso

        xSemaphoreGive(xMutex);          // Libera el mutex

        vTaskDelay(pdMS_TO_TICKS(500)); // Espera antes de
repetir
    }

}

// Tarea de BAJA prioridad (bloquea el recurso)
void low_priority_task(void *pvParameters) {

    while (1) {

        printf("[BAJA] Tomando el mutex...\n");

        xSemaphoreTake(xMutex, portMAX_DELAY); // Bloquea el
recurso

        printf("[BAJA] Recurso en uso...\n");

        vTaskDelay(pdMS_TO_TICKS(1000)); // Retraso intencional
(causa inversión)

        xSemaphoreGive(xMutex);          // Libera el mutex

        printf("[BAJA] Mutex liberado.\n");

        vTaskDelay(pdMS_TO_TICKS(2000)); // Espera larga

    }

}

void app_main() {

    // Crear mutex con prioridad heredada

    xMutex = xSemaphoreCreateMutex();

    xQueue = xQueueCreate(5, sizeof(int));

    // Crear tareas

```

```

    xTaskCreate(high_priority_task, "Alta", 2048, NULL, 3, NULL);
// Prioridad 3 (alta)

    xTaskCreate(low_priority_task, "Baja", 2048, NULL, 1, NULL);
// Prioridad 1 (baja)

// Iniciar scheduler

vTaskStartScheduler();

}

```

9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.

La herencia de prioridad es un mecanismo utilizado en sistemas de tiempo real (RTOS) para resolver el problema de inversión de prioridad, donde una tarea de alta prioridad queda bloqueada indefinidamente porque una tarea de baja prioridad retiene un recurso compartido.

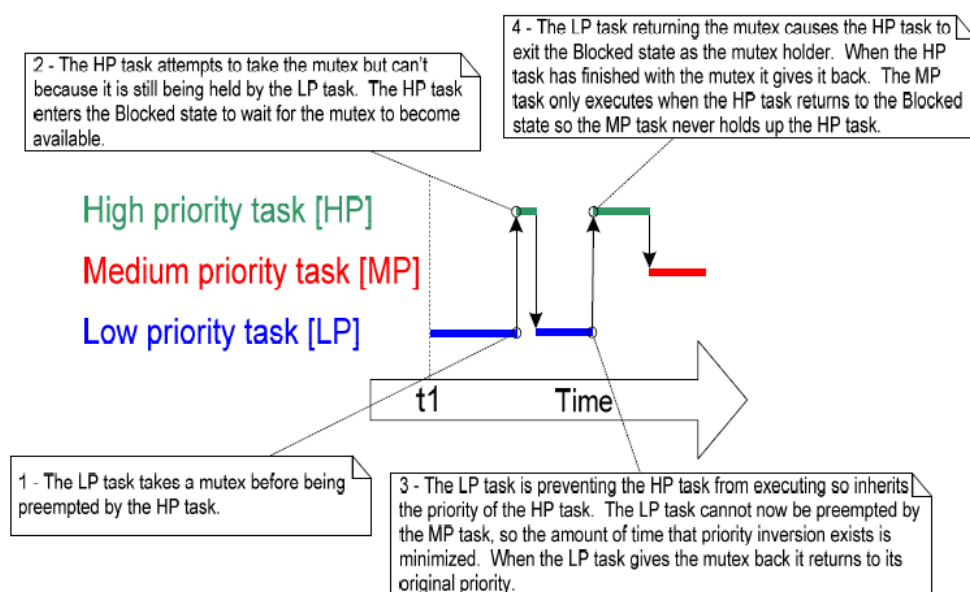


Figure 40 Priority inheritance minimizing the effect of priority inversion

10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

```

#include <FreeRTOS.h>

#include <task.h>

#include <semphr.h>

```

```

#include <stdio.h>

// Recursos compartidos (simulan periféricos como SPI/UART)
SemaphoreHandle_t xMutexUART; // Recurso 1: UART
QueueHandle_t xQueue;         // Cola para mensajes

// Tarea 1 (Prioridad media)
void vTask1(void *pvParameters) {
    while (1) {
        printf("Tarea 1: Intentando tomar UART...\n");

        xSemaphoreTake(xMutexUART, portMAX_DELAY); // Bloquea
UART

        printf("Tarea 1: UART tomado. Intentando tomar SPI...\n");

        vTaskDelay(pdMS_TO_TICKS(10)); // Pequeño retraso
(aumenta probabilidad de deadlock)

        printf("Tarea 1: Recurso tomado. Procesando...\n");

        xSemaphoreGive(xMutexUART);

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

// Tarea 2 (Prioridad alta)
void vTask2(void *pvParameters) {
    while (1) {

```

```
        xSemaphoreTake(xMutexUART, portMAX_DELAY); // ;Bloqueado
si Tarea 1 ya tomó UART!
```

```
    printf("Tarea 2: Recurso tomado. Procesando...\n");
```

```
    xSemaphoreGive(xMutexUART);
```

```
    vTaskDelay(pdMS_TO_TICKS(1000));
```

```
}
```

```
}
```

```
void app_main() {
```

```
    // Crear mutex para recursos
```

```
    xMutexUART = xSemaphoreCreateMutex();
```

```
    xQueue = xQueueCreate(5, sizeof(int));
```

```
    // Crear tareas
```

```
    xTaskCreate(vTask1, "Task1", 2048, NULL, 2, NULL); //
Prioridad 2
```

```
    xTaskCreate(vTask2, "Task2", 2048, NULL, 3, NULL); //
Prioridad 3 (más alta)
```

```
    // Iniciar scheduler
```

```
    vTaskStartScheduler();
```

```
}
```